- That’s just 4% of the entire month when the released version of the product had a problem.

Pretty good.

As a counter example, let’s take a team that releases their software every two weeks with 1 day of regression testing and a 4-hour release at the end of their development cycle. To complete 10 releases would take 10 full days of regression testing and an additional 5 days of release activity, and that’s not even measuring the time it would take to find problems and fix code. That would mean that over half of their time was used fixing bugs no matter what the CLOC was on the fixes.

Realistically there are potential outcomes in these scenarios:

- Teams don’t fix bugs because they know they won’t be able to complete any more features if they spend all of their time testing and releasing code.
- Teams jam as many fixes as possible into a release, and thus you’ll see a very high CLOC on hot fixes. This results in longer test cycles and bigger releases, both of which are indicators of code with poor maintainability.

To make sure CLOC and number of fixes are relevant, ensure that you’re also counting total deployments so you can get a percentage of fix to release (FRP). To summarize, the formula for that is

```
Fix Release Percentage (FRP) = Total fixes / releases
```

You want your fix release percentage to be as low as possible. A 0 is perfect and anything over .5 is usually pretty bad. By combining MTTR and FRP you can calculate a maintainable release rating with this formula:

```
Maintainable Release = MTTR(in minutes) * (Total Fixes / Releases)
```

In this case the closer to 0 you are the better. Continuing the initial example, if you have 60 releases and 10 fixes, you have a 16% FRP. If the same team